Ignytion io

IGNYTION IO · TECHNICAL LAB NOTE

Insert and Verify an Eight-Bit Scan Chain

OpenROAD scan replacement, serial shift, and functional capture with SKY130

A scan chain lets a test sequence load and observe the state held in flip-flops. This note starts with an eight-bit writable register, maps its logic to SKY130 cells, and uses OpenROAD to insert scan. The inserted gate netlist is tested with every byte value. A disconnected scan link demonstrates that the same checker can reject a fault.

SECTION 01

What passed

OpenROAD replaced **eight ordinary flip-flops** with scan cells and stitched one eight-bit chain. The inserted gate netlist passes the functional, serial-shift, and capture tests.

CHECK	MEASURED RESULT	STATE
Baseline RTL	256 byte patterns · 514 checks	✓ PASS
Inserted gate netlist	256 byte patterns · 3,330 checks	✓ PASS
Chain structure	1 chain · 8 scan cells	✓ PASS
Broken internal link	Pattern 1 fails · simulator exit 1	✓ PASS

A functional scan demonstration

The test uses the official library's **functional models with zero unit delay**. It does not establish ATPG fault coverage, scan timing closure, routed implementation quality, or manufacturing-test qualification.

Crucible is beta. The project environment binds Yosys, OpenROAD, and Icarus Verilog. The PDK libraries remain explicit local dependencies; their mere presence does not establish a working insertion flow.

SECTION 02

Shift and capture serve different purposes

A **scan flip-flop** can select functional data or serial test data at its input. A common scan-enable signal selects the mode.

MODE	DATA PATH	CLOCK ACTION
Functional	Normal register logic	Capture on rising edge
Shift	Previous scan cell or scan_in	Move one bit per rising edge
Capture	Normal logic · scan disabled	Observe the functional response

The register has a **synchronous active-low reset**. Reset clears all bits on a rising edge. With reset inactive, write enable loads the input byte; otherwise the register holds its previous value.

During shift, reset is inactive and write enable is low. Scan enable overrides the normal data selection inside each inserted scan cell. After loading a byte, the test disables scan and applies a normal capture edge.

> **What the scan output is here**
> The chain tail already drives an observable output, **q[0]**. OpenROAD reuses that functional output bit as the serial observation point. Only scan_in and scan_enable are added; there is no separate scan_out port in the saved netlist.

SECTION 03

Run insertion on a mapped design

Yosys maps the register to ordinary SKY130 flip-flops and combinational cells. OpenROAD reads the technology and cell views, replaces flops, and stitches after placement.

```
set_dft_config -max_length 8 -max_chains 1 \
    -clock_mixing no_mix \
    -scan_enable_name_pattern scan_enable \
    -scan_in_name_pattern scan_in \
    -scan_out_name_pattern scan_out
scan_replace
global_placement -skip_io -density 0.30
report_dft_plan -verbose
execute_dft_plan
```

These lines occur in the executed insertion script. The saved **Verilog, OpenDB, and DEF** retain the result. The DEF is a placed-stage artifact; it is not a routed or signoff result.

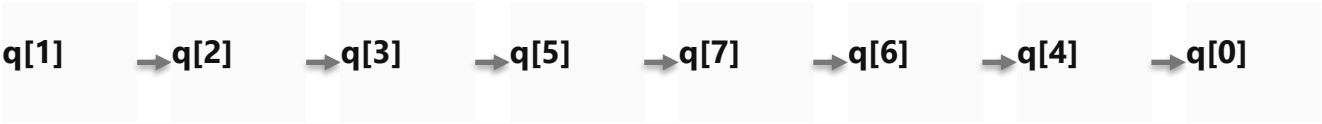
The local technology LEF and cell LEF share a basename. Importing both under the same inferred library name initially collided. A byte-identical technology-LEF copy with a distinct basename preserves the site definitions and lets both views load.

SECTION 04

Read the actual chain order

The serial order is derived from **SCD-to-Q connectivity** in the inserted netlist. It is not assumed to follow ascending RTL bit indices.

INSERTED CHAIN · HEAD TO TAIL



scan_in feeds the leftmost stage. On a shift edge, each bit moves towards q[0] on the right. The test samples q[0] before the edge so the previous tail value is observed.

CHECKED STRUCTURE	OBSERVATION
Head	Exactly one scan cell is driven by scan_in
Internal links	Every Q drives exactly the next SCD
Coverage of register	All eight distinct q bits occur once
Tail	q[0] is an existing top-level output

The chain parser generates a testbench mapping from these connections. Serial load sends the requested byte in reverse chain order so that eight shift edges leave the intended value on q.

SECTION 05

Verify every byte value

For each of **256 values**, the same testbench exercises load and hold. The gate-level run adds serial unload, serial load, and a return to functional capture.

STEP	OBSERVATION
Reset	q is zero after a reset edge
Functional load	q equals the requested byte
Hold	Changing data without write leaves q unchanged
Serial unload	Eight observed tail bits match captured data
Serial load	Eight injected bits reconstruct the target byte
Functional capture	Scan disabled · complement captured correctly

```
DFT_BASELINE_PASS patterns=256 checks=514
DFT_SCAN_CAPTURE_PASS patterns=256 checks=3330
```

Each comparison uses four-state equality, so an unknown value fails. A watchdog turns a stalled run into a fatal error. The observer requires these success records and also checks that the positive logs contain no fatal error or mismatch.

SECTION 06

Make the checker reject a broken chain

The negative control changes **one internal scan-data connection** to constant zero. It leaves the functional circuit and testbench intact.

```
FATAL: tests/tb.v:26:
Q_MISMATCH pattern=1 got=00 expected=01
Time: 572000  Scope: tb.check_q
Simulator exit: 1
```

The failure occurs after a serial load. Pattern 1 should reconstruct 0x01, but the interrupted path loses its set bit and produces 0x00. This is a causal observation of the injected scan defect.

The runner saves the simulator's nonzero exit separately. The verification script rejects a negative run that exits successfully, lacks the expected mismatch, or prints the positive scan-pass record.

> **This is not an ATPG coverage figure**
> Enumerating every register byte checks this small shift/capture contract. It does not enumerate all stuck-at or transition faults in the combinational logic, and it supplies no manufacturing fault-coverage percentage.

SECTION 07

Reproduce and inspect the evidence

Run from the **DFT_Scan workspace in WSL**. The scripts select the project environment and use the installed local SKY130 HD views.

```
igny init
bash scripts/run.sh build
bash scripts/run.sh test
python3 scripts/verify.py
DFT_INSERTION_FUNCTIONAL_SCAN_AND_NEGATIVE_CONTROL_VERIFIED
```

FILE	PURPOSE
reports/insertion.log	Actual scan plan and tool completion
reports/chain.json	Derived serial order and pin connections
reports/inserted.log	Positive gate-level comparisons
reports/disconnected.log	Injected defect and fatal mismatch
reports/simulation_models.json	Model origin, modules, and hashes
reports/summary.json	Checked outcome and artifact hashes

The full PDK Verilog aggregate contains an unrelated malformed module. Icarus preprocessing selects the functional branches; the script then extracts the used modules and their dependencies unchanged. **No behavioural scan model was invented** for this test.

SECTION 08

Limits and source notes

This experiment verifies **one clock, one eight-bit chain, and one synchronous-reset register**. Its result is a working local insertion and simulation example.

It does not cover ATPG, stuck-at or transition-fault coverage, multi-clock chains, scan-clock distribution, JTAG, test compression, power-aware simulation, routing, shift/capture STA, physical-rule checking, or manufactured devices. The nominal clock constraint is an implementation input, not proof of a supported frequency.

SOURCE NOTES

OpenROAD · Design for Testing
`https://openroad.readthedocs.io/en/latest/main/src/dft/README.html`

SKY130 high-density standard cells
`https://skywater-pdk.readthedocs.io/en/main/contents/libraries/sky130_fd_sc_hd/README.html`

Yosys documentation
`https://yosyshq.readthedocs.io/projects/yosys/en/latest/`

Icarus Verilog documentation
`https://steveicarus.github.io/iverilog/`

All numerical results are from the local run on **27 October 2026**. The source and gate-netlist hashes accompany the reports. Tool and PDK authors retain credit; no endorsement is implied.